Assignment 2

Q1) Reverse a string using stack.

```
#include < stdio.h>
# include < string.h>
# define Max 100
    char stack [max]
        int top = -1
  Void push (char c){
            if (top = = Max-1){
              printf("stack overload \n");
              return;
            }
            stack [++top] = c;
      }
        char pop(){
              if (top = = -1){
              printf("stack underflow \n");
              return '\0';
        }
          return stack [top--];
  int main(){
        char str [MAX]
        printf ("Enter the string:");
        scanf ("%s", str);
        for (int i=0; i< strlen (str); i++){
                  push(str[i]);
        }
          printf ("Reversed string:");
          while (top ≠ -1){
                  printf ("%c", pop());
          }
              printf ("\n");
              return 0;
        }
```

Q2) Balanced parenthesis

```
#include <stdio.h>
# include <string.h>
# define max 100
    char stack [MAX]
    int top=-1
    void push (char c){
      if (top == MAX-1){
        printf ("stack over flow \n");
        return;
      }
      stack [++top]=c
    }
    char pop (){
        if (top==-1){
        } return '\0';
      return stack [top--];
    }
    int isMatching [char left, char right){
    if (left=='(' && right==')') return 1;
    if (left=='[' && right==']') return 1;
    if (left=='{' && right=='}') return 1;

int Balanced (char *expr){
    for (int i=0; i<strlen (expr); i++){
      if (expr[i] =='(' || expr[i]=='{' ||
            expr[i] =='[') {
          push (expr[i]);
    } else if (expr[i] ==')' || expr[i]=='}' ||
            expr[i]==']') {
          char popped = pop()
```

```
        if(! is matching (popped,
            return 0,
    }
  }
}
}   return (top==-1)
int main(){
    char expr[MAX]
    printf("Enter an expression);
    scanf("%s", expr);
    if (Balanced (expr)){
        printf("Balanced Expression \n");
    } else {
        printf("Not Balanced expression \n");
    } return 0;
}
```

Q3) Next Greater Element:-

```
#include <stdio.h>
#include MAX 100
int stack [MAX];
int top = -1;
void push (int x){
    if (top == MAX -1){
    printf("stack over flow \n");
    return;
    }
        stack [++top] = x;
    }
int pop (){
    if (top == -1){
        return -1;
    }
```

```
return stack [top--];
}
int peak(){
if (top==-1) return -1;
return stack [top];
}
void next greater [int arr[], int n){
        int result[n];
  for (int i=0; i<n; i++){
~~void~~        result [i]= -1;
  }
  for (int i=0; i<n; i++){
  while (top≠-1 && arr[i] >arr [peak()]){
        int index = pop();
        result [index]=arr[i];
  }
    Push (i);
  }
  for (int i=0; i<n; i++){
  printf("%d -> %d \n", arr[i], result [i]);
  }
}
int main(){
  int arr[] = { 4, 5, 6, 10, 8};
  int n= size of (arr) (size of [arr[0]);
  next-greater (arr, n);
  return 0;
}
```

Q4) printer queue simulation:

```
#include <stdio.h>
#include <string.h>
#define MAX 100
char queue[MAX][50];
int front = -1, rear = -1;
void add Doctument (char* doc){
    if (rear == MAX-1){
    printf("queue over flow\n");
    return;
    }
    if (front == -1) front = 0;
    strcpy (queue[++rear], doc);
    Print ("Document \"%s\" added to queue\n",
                                doc);
}

void print Document (){
        if (front == -1 || front > rear){
            Print("no documents to print.\n");
            return;
        }
    printf("printing document: %s\n", queue[front]);
    front ++;
}
void display queue (){
        if (front == -1 || front > rear){
        Printf("No pending documents.\n");
        return;
    }
    printf("pending documents:\n");
    for (int i = front; i <= rear; i++){
        Printf("%d. %s\n"
```

```
}
int main (){
    int choice;
    char doc[50];
    while(1){
      printf("\n--- Printer queue menu --\n");
      printf("1. Add Document \n");
      printf("2. print document \n");
      printf("3. Display pendings \n");
      printf("4. exit \n");
printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice){
        case 1:
            printf("Enter document name ");
            scanf("%s", doc);
            add document (doc);
            break;
        case 2:
            print Document ();
            break;
        case 3:
            display queue ();
            break;
        case 4:
            printf("Exiting ...\n");
            return 0;
        default:
            printf("Invalid choice ! ");
    }
}
}
```

Circular Queue Basic operations:-

```
#include <stdio.h>
#define MAX 5
int queue [MAX]
int front = -1, rear = -1;
int Full () {
    return ((rear+1) % MAX == front);
}
int isEmpty () {
    return (front == -1);
}
Void enqueue (int x) {
    if (isfull ()) {
        printf ("queue overflow:", x);
        return;
    }
    if (isfull()) {
    if (isEmpty ()) {
        front = rear = 0;
    }
    else {
        rear = (rear+1) % MAX;
    }
    queue [rear] = x;
    printf ("%d "inserted into queue.\n", x);
}
Void enqueue (int 2) {
Void dequeue () {
    if (isEmpty()) {
        Printf ("queue underflow \n");
        return;
    }
```

```
Printf ("Removed element : %d \n", queue[front])
    if (front == rear){
        front = rear = -1;
    }
    else{
        front = (front + 1) % MAX;
    }
void peek (){
    if (isEmpty () {
    printf("Queue is empty \n");
    return;
printf ("Queue elements : ");
  int i = front;
  while (1){
    Printf ("%d", queue [i]));
    if (i == rear) break;
    r = (i+1) % max
  }
  print ("\n");
}
int main(){
    int choice, value;
    while (1){
    printf ("\n.. circular queue ... \n");
    printf ("1. Enqueue \n");
    Printf ("2. Dequeue\n");
    Printf ("3. PEEK \n");
    Printf ("4. Display \n");
    printf (" 5. Exit \n");
    printf("Enter your choice:");
    scanf ("%d", &choice);
  switch (choice){
```

```
        printf("Enter value to insert:");
        scanf("%d", &value);
        enqueue(value);
        break;
    case 2:
        dequeue();
        break;
    case 3:
        peek();
        break;
    case 4:
        display();
        break
    case 5:
        printf("Exiting...\n");
        return 0;
    default;
        printf("Invalid choice!");
```